



程序设计实习

郭 炜

微博: <http://weibo.com/guoweiofpku>

学会程序和算法，走遍天下都不怕!

讲义照片均为郭炜拍摄



北京大学
PEKING UNIVERSITY

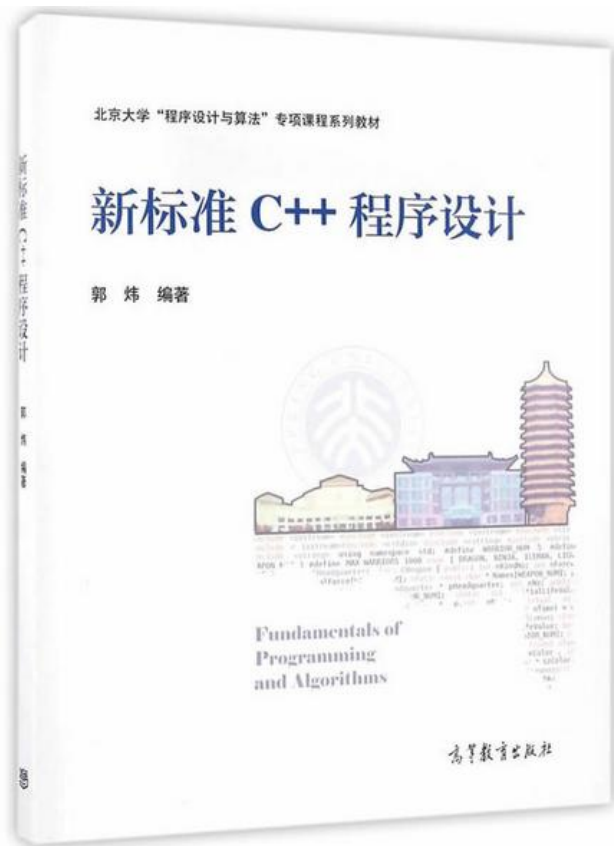
信息科学技术学院

配套教材：

高等教育出版社

《新标准C++程序设计》

郭炜 编著

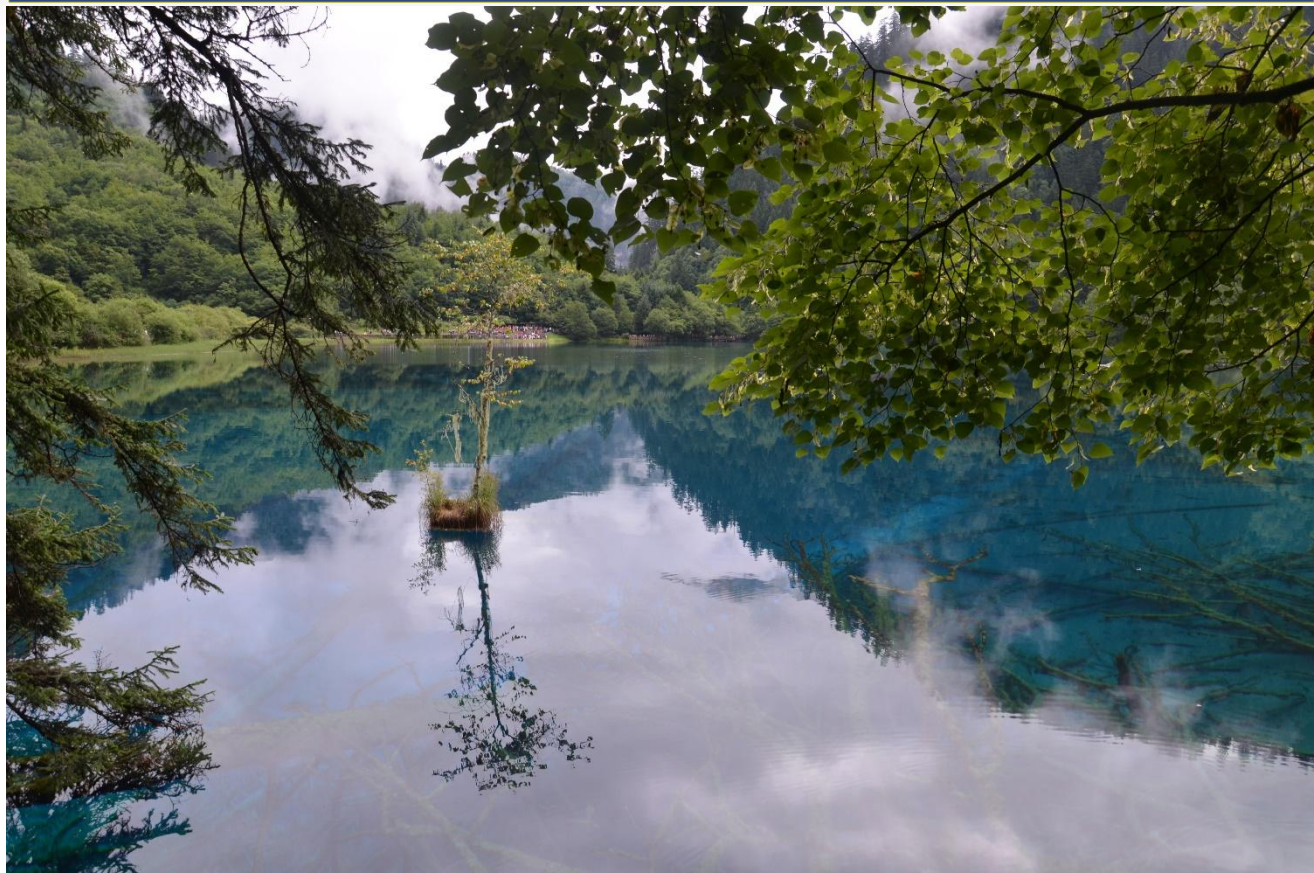




北京大学
PEKING UNIVERSITY

信息科学技术学院 郭炜

C++11特性



九寨沟

统一的初始化方法

- 可以统一用 { } 进行初始化

```
int arr[3]{1, 2, 3};
vector<int> iv{1, 2, 3};
map<int, string> mp{{1, "a"}, {2, "b"}};
string str{"Hello World"};
int * p = new int[20]{1,2,3};
struct A {
    int i,j;
};
A func(int m,int n ) { return {m,n}; }
int main() {
    A a1{10,20},a2(100,200),b; //A有默认构造函数和双参数构造函数
    A * pa = new A {3,7};
    A a[] = {{1,2},{3,4},{5,6}};
}
```

成员变量默认初始值

```
class B
{
    public:
        int m = 1234;
        int n;
};

int main()
{
    B b;
    cout << b.m << endl;    //输出 1234
    return 0;
}
```

auto关键字

用于定义变量，编译起可以自动判断变量的类型

```
auto i = 100;           // i 是 int
auto p = new A();       // p 是 A *
auto k = 34343LL;       // k 是 long long
```

```
map<string,int,greater<string> > mp;
for( auto i = mp.begin(); i != mp.end(); ++i)
    cout << i->first << "," << i->second ;
//i的类型是:  map<string,int,greater<string> >::iterator
```

auto关键字

```
class A { };  
  
A operator + ( int n,const A & a)  
{  
    return a;  
}  
  
template <class T1, class T2>  
auto add(T1 x, T2 y) -> decltype(x + y) {  
    return x+y;  
}  
  
auto d = add(100,1.5); // d是double d=101.5  
auto k = add(100,A()); // d是A类型
```

decltype 关键字

求表达式的类型

```
int i;  
double t;  
struct A { double x; };  
const A* a = new A();
```

```
decltype(a)      x1;    // x1 is A *  
decltype(i)      x2;    // x2 is int  
decltype(a->x)    x3;    // x3 is double  
decltype((a->x))  x4 = t; // x4 is double&
```


智能指针shared_ptr

- 头文件: `<memory>`
- 通过shared_ptr的构造函数, 可以让shared_ptr对象托管一个new运算符返回的指针, 写法如下:
- `shared_ptr<T> ptr(new T);` // T 可以是 int ,char, 类名等各种类型
此后ptr就可以像 `T*` 类型的指针一样来使用, 即 `*ptr` 就是用new动态分配的那个对象, 而且不必操心释放内存的事。
- 多个shared_ptr对象可以同时托管一个指针, 系统会维护一个托管计数。当无shared_ptr托管该指针时, delete该指针。
- shared_ptr对象不能托管指向动态分配的数组的指针, 否则程序运行会出错

智能指针shared_ptr

```
#include <memory>
#include <iostream>
using namespace std;

struct A {
    int n;
    A(int v = 0):n(v){ }
    ~A() { cout << n << " destructor" << endl; }
};

int main()
{
    shared_ptr<A> sp1(new A(2)); //sp1托管A(2)
    shared_ptr<A> sp2(sp1);      //sp2也托管 A(2)
    cout << "1)" << sp1->n << ", " << sp2->n << endl;
    //输出1) 2,2
    shared_ptr<A> sp3;
    A * p = sp1.get();          //p 指向 A(2)
    cout << "2)" << p->n << endl;
```

输出结果:

1) 2, 2

2) 2

```

sp3 = sp1;    //sp3也托管 A(2)
cout << "3)" << (*sp3).n << endl;    //输出 2
sp1.reset();    //sp1放弃托管 A(2)
if( !sp1 )
    cout << "4)sp1 is null" << endl;    //会输出
A * q = new A(3);
sp1.reset(q);    // sp1托管q
cout << "5)" << sp1->n << endl;    //输出 3
shared_ptr<A> sp4(sp1);    //sp4托管A(3)
shared_ptr<A> sp5;
sp1.reset();    //sp1放弃托管 A(3)
cout << "before end main" << endl;
sp4.reset();    //sp1放弃托管 A(3)
cout << "end main" << endl;
return 0;    //程序结束, 会delete 掉A(2)

```

```

}
```

输出结果:

```

1) 2, 2
2) 2
3) 2
4) sp1 is null
5) 3
before end main
3 destructor
end main
2 destructor

```

智能指针shared_ptr

```
#include <iostream>
#include <memory>
using namespace std;
struct A{
    ~A() { cout << "~A" << endl; }
};
int main()
{
    A * p = new A();
    shared_ptr<A> ptr(p);
    shared_ptr<A> ptr2;
    ptr2.reset(p);    //并不增加 ptr中对p的托管计数
    cout << "end" << endl;
    return 0;
}
```

智能指针shared_ptr

```
#include <iostream>
#include <memory>
using namespace std;
struct A{
    ~A() { cout << "~A" << endl; }
};
int main()
{
    A * p = new A();
    shared_ptr<A> ptr(p);
    shared_ptr<A> ptr2;
    ptr2.reset(p);    //并不增加 ptr中对p的托管计数
    cout << "end" << endl;
    return 0;
}
```

输出:

end

~A

~A

之后程序崩溃

因 p被delete两次

智能指针unique_ptr

- 只需要一个指针管理对象
→ 用 unique_ptr (更快、更安全)
- 需要多个指针共用对象
→ 用 shared_ptr

表格		
特性	unique_ptr	shared_ptr
所有权	独占	共享
拷贝	不允许拷贝	允许拷贝, 计数 + 1
转移	使用 <code>std::move</code>	直接拷贝即可
内存开销	极小	额外存引用计数
性能	更快	稍慢 (计数原子操作)
释放时机	自身销毁时立即释放	计数为 0 时释放
循环引用	无问题	会内存泄漏, 需搭配 <code>weak_ptr</code>

智能指针unique_ptr

```
#include <iostream>
#include <memory> // for unique_ptr
int main() {
    // 1. Create unique_ptr
    std::unique_ptr<int> p1(new int(100));
    // 2. Cannot copy
    // unique_ptr<int> p2 = p1; // ERROR
    // 3. Can move ownership
    std::unique_ptr<int> p2 = std::move(p1);
    if (!p1) {
        std::cout << "p1 is now null\n"; //会输出
    }
    // 4. Access value
    std::cout << *p2 << "\n"; //>>100
    // 5. Automatically delete when out of scope
    return 0;
}
```

空指针nullptr

```
#include <memory>
#include <iostream>
using namespace std;

int main()    {
    int* p1 = NULL;
    int* p2 = nullptr;
    shared_ptr<double> p3 = nullptr;
    if(p1 == p2)
        cout << "equal 1" <<endl;
    if( p3 == nullptr)
        cout << "equal 2" <<endl;
    if( p3 == p2) ; // error
    if( p3 == NULL)
        cout << "equal 4" <<endl;
    bool b = nullptr; // b = false
    int i = nullptr; //error,nullptr不能自动转换成整型
    return 0;
}
```

去掉出错的语句后输出：

equal 1

equal 2

equal 4

基于范围的for循环

```
#include <iostream>
#include <vector>
using namespace std;

struct A {    int n; A(int i):n(i) {    } };

int main() {
    int ary[] = {1,2,3,4,5};
    for(int & e: ary)
        e*= 10;
    for(int e : ary)
        cout << e << ",";
    cout << endl;
    vector<A> st(ary,ary+5);
    for( auto & it: st)
        it.n *= 10;
    for( A it: st)
        cout << it.n << ",";
    return 0;
}
```

输出:

10,20,30,40,50,
100,200,300,400
,500,

右值引用和move语义

右值：一般来说，不能取地址的表达式，就是右值，
能取地址的，就是左值

```
class A { };
```

```
A & r = A(); // error , A()是无名变量，是右值
```

```
A && r = A(); //ok, r 是右值引用
```

主要目的是提高程序运行的效率，减少需要进行深拷贝的对象进行深拷贝的次数。

[参考](#)

<http://amazingjxq.com/2012/06/06/%E8%AF%91%E8%AF%A6%E8%A7%A3c%E5%8F%B3%E5%80%BC%E5%BC%95%E7%94%A8/>

<http://www.cnblogs.com/soaliap/archive/2012/11/19/2777131.html>

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

class String
{
public:
    char * str;
    String():str(new char[1]) { str[0] = 0;}
    String(const char * s) {
        str = new char[strlen(s)+1];
        strcpy(str,s);
    }
    String(const String & s) {
        cout << "copy constructor called" << endl;
        str = new char[strlen(s.str)+1];
        strcpy(str,s.str);
    }
}
```

```
String & operator=(const String & s) {  
    cout << "copy operator= called" << endl;  
    if( str != s.str) {  
        delete [] str;  
        str = new char[strlen(s.str)+1];  
        strcpy(str,s.str);  
    }  
    return * this;  
}  
  
// move constructor  
String(String && s):str(s.str) {  
    cout << "move constructor called"<<endl;  
    s.str = new char[1];  
    s.str[0] = 0;  
}
```

```
// move assignment
```

```
String & operator = (String &&s) {  
    cout << "move operator= called"<<endl;  
    if (str!= s.str) {  
        delete [] str;  
        str = s.str;  
        s.str = new char[1];  
        s.str[0] = 0;  
    }  
    return *this;  
}  
~String() { delete [] str; }  
};
```

```
template <class T>
void MoveSwap(T& a, T& b)    {
    T tmp(move(a)); // std::move(a) 为右值, 这里会调用move
    constructor
    a = move(b); // move(b) 为右值, 因此这里会调用move assignment
    b = move(tmp); // move(tmp) 为右值, 因此这里会调用move
    assignment
}
```

```
int main()
{
    //String & r = String("this"); // error
    String s;
    s = String("ok"); // String("ok")是右值
    cout << "*****" << endl;
    String && r = String("this");
    cout << r.str << endl;
    String s1 = "hello", s2 = "world";
    MoveSwap(s1, s2);
    cout << s2.str << endl;
    return 0;
}
```

输出:

move operator= called

this

move constructor called

move operator= called

move operator= called

hello

函数返回值为对象时，返回值对象如何初始化？

➤只写复制构造函数

return 局部对象 -> 复制

return 全局对象 -> 复制

➤只写移动构造函数

return 局部对象 -> 移动

return 全局对象 -> 默认复制

return move(全局对象) -> 移动

➤同时写 复制构造函数和 移动构造函数:

return 局部对象 -> 移动

return 全局对象 -> 复制

return move(全局对象) -> 移动

所以不必担心return 一个局部 vector对象会导致复制整个数组耗费时间
dev c++中，return 局部对象 会导致优化，不调用移动或复制构造函数

可移动但不可复制的对象:

```
struct A{
    A(const A & a) = delete;
    A(const A && a) { cout << "move" << endl; }
    A() { };
};
A b;
A func() {
    A a;
    return a;
}
void func2(A a) { }
int main() {
    A a1;
    A a2(a1);      //compile error
    func2(a1);     //compile error
    func();
    return 0;
}
```

无序容器(哈希表)

```
#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;
int main()
{
    unordered_map<string,int> turingWinner; //图灵奖获奖名单
    turingWinner.insert(make_pair("Dijkstra",1972));
    turingWinner.insert(make_pair("Scott",1976));
    turingWinner.insert(make_pair("Wilkes",1967));
    turingWinner.insert(make_pair("Hamming",1968));
    turingWinner["Ritchie"] = 1983;
    string name;
    cin >> name; //输入姓名
```

```
        unordered_map<string,int>::iterator p =
turingWinner.find(name);
        //据姓名查获获奖时间
        if( p != turingWinner.end())
            cout << p->second;
        else
            cout << "Not Found" << endl;
        return 0;
    }
```

哈希表插入和查询的时间复杂度几乎是常数

正则表达式

```
#include <iostream>
#include <regex> //使用正则表达式须包含此文件
using namespace std;
int main()
{
    regex reg("b.?p.*k");
    cout << regex_match("bopggk", reg) << endl; //输出 1, 表示匹配成功
    cout << regex_match("boopgggk", reg) << endl; //输出 0, 匹配失败
    cout << regex_match("b pk", reg) << endl; //输出 1, 表示匹配成功
    regex reg2("\\d{3}([a-zA-Z]+). (\\d{2}|N/A)\\s\\1");
    string correct="123Hello N/A Hello";
    string incorrect="123Hello 12 hello";
    cout << regex_match(correct, reg2) << endl; //输出 1, 匹配成功
    cout << regex_match(incorrect, reg2) << endl; //输出 0, 失败
}
```

Lambda表达式

只使用一次的函数对象，能否不要专门为其编写一个类？

只调用一次的简单函数，能否在调用时才写出其函数体？

Lambda表达式

形式:

[外部变量访问方式说明符](参数表) -> 返回值类型

```
{  
    语句组  
}
```

[] 不使用任何外部变量

[=] 以传值的形式使用所有外部变量

[&] 以引用形式使用所有外部变量

[x, &y] x 以传值形式使用, y 以引用形式使用

[=, &x, &y] x, y 以引用形式使用, 其余变量以传值形式使用

[&, x, y] x, y 以传值的形式使用, 其余变量以引用形式使用

“->返回值类型” 也可以没有, 没有则编译器自动判断返回值类型。

Lambda表达式

```
int main()
{
    int x = 100,y=200,z=300;
    cout << [ ](double a,double b) { return a + b; }(1.2,2.5)
        << endl;
    auto ff = [=,&y,&z](int n) {
        cout <<x << endl;
        y++; z++;
        return n*n;
    };
    cout << ff(15) << endl;
    cout << y << "," << z << endl;
}
```

Lambda表达式

```
int main()
{
    int x = 100,y=200,z=300;
    cout << [ ](double a,double b) { return a + b; } (1.2,2.5)
        << endl;
    auto ff  = [=,&y,&z](int n) {
        cout <<x << endl;
        y++; z++;
        return n*n;
    };
    cout << ff(15) << endl;
    cout << y << ", " << z << endl;
}
```

输出:

3.7

100

225

201,301

Lambda表达式

```
int a[4] = { 4,2,11,33};  
sort(a,a+4,[ ](int x,int y)->bool {  
    return x%10 < y%10; } );  
  
for_each(a,a+4,[ ](int x) {cout << x << " " ;} ) ;
```

Lambda表达式

```
int a[4] = { 4,2,11,33};  
sort(a,a+4,[ ](int x,int y)->bool { return x%10  
                                     < y%10; });  
for_each(a,a+4,[ ](int x) {cout << x << " ";} ) ;
```

输出:
11 2 33 4

Lambda表达式

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int main()
{
    vector<int> a { 1,2,3,4};
    int total = 0;
    for_each(a.begin(),a.end(), [&](int & x)
        {total += x; x*=2;});
    cout << total << endl; //输出 10
    for_each(a.begin(),a.end(),[ ](int x)
        { cout << x << " ";});
    return 0;
}
```

程序输出结果:

10

2 4 6 8

Lambda表达式

实现递归求斐波那契数列第n项:

```
#include <functional>
```

```
function<int(int)> fib = [&fib](int n)
```

```
{ return n <= 2 ? 1 : fib(n-1) + fib(n-2);};
```

```
cout << fib(5) << endl;    //输出5
```

`function<int(int)>` 表示返回值为 `int`, 有一个`int`参数的函数

lambda表达式本质

➤ lambda表达式是函数对象（仿函数）的语法糖

```
int main() {  
    int x=10, y=10;  
    int f2 = [x,&y] (int a) {  
        y += 10;  
        return a + x + y;  
    };  
    cout << f2(100) << endl; //>>130  
    cout << y << endl; //>>20  
    return 0;  
}
```

lambda表达式本质

➤ 等价于:

```
int main() {  
    int x=10, y=10;  
    struct f2_type {  
        int x;  
        int & y;  
        f2_type(int xx,int & yy)  
        :x(xx),y(yy) {}  
        int operator()(int a) const {  
            y += 10;  
            return a + x + y;  
        }  
    };  
    f2_type f2(x,y);  
    cout << f2(100) << endl; //>>130  
    cout << y << endl;      //>>20  
    return 0;  
}
```

模板类型参数包和递归

► 模版的类型参数可以有任意多个，模版可以递归

template<typename... Ts> // Ts是类型参数包，可以有任意多个类型
struct Tuple; //可以存放任意多个不同类型的数据的序列

// 递归特化：拆出第一个类型，剩下的继续递归

```
template<typename T, typename... Ts>
struct Tuple<T, Ts...> {
    T value;
    Tuple<Ts...> rest;
    Tuple(T v, Ts... rest_args)
        : value(v), rest(rest_args...) { }
};
```

template<> //递归终止：空参数包

```
struct Tuple<> {};
```

```
int main() {
    Tuple<int, double, const char*> t{10, 3.14, "hello"};
    std::cout << t.value << ", " << t.rest.value << ", "
    << t.rest.rest.value; //10,3.14,hello,
}
```

万能引用和forward转发

- 函数模板的形参如果是 && 类型，称为万能引用
- 实例化时可根据实参自动变为左值或右值

```
template<class T>
void f( T && x) { //参数为万能引用
    //既能接收左值，也能接收右值，而且保留原本的值类别（左值 / 右值）。
    x = 20;
    cout << x << endl;
}

int main() {
    f(123); //>>20      实例化出void f(int &&)
    int a = 8;
    f(a); //>>20      实例化出 void f(int & )
    cout << a << endl; //>>20
    return 0;
}
```


万能引用和forward转发

- 函数模板的形参如果是 **T &&** 类型，称为万能引用
- 实例化时可根据实参自动变为左值或右值

#include <utility> // **std::forward** 所在头文件

```
void process(int& lval) { std::cout << "left:" << lval << std::endl; }
void process(int&& rval){ std::cout << "right: " << rval << std::endl;}
template<typename T>
void wrapper(T&& arg) { // 自动左右值适配 (万能引用 T&&)
    // 如果 arg 是右值, std::forward 将其转为右值, 调用 process(int&&)
    // 如果 arg 是左值, std::forward 将其保持为左值, 调用 process(int&)
    process(std::forward<T>(arg));
    //若写 process(arg);则一概调用 process(int&)
}

int main() {
    int x = 10;
    wrapper(x);           // 传入左值 -> 调用 process(int&)
    wrapper(20);          // 传入右值 -> 调用 process(int&&)
    return 0;
}
```

多线程

```
#include <iostream>
#include <thread>
using namespace std;
struct MyThread {
    void operator () () {
        while(true)
            cout << "IN MYTHREAD\n";
    }
};
void my_thread(int x)
{
    while(x)
        cout << "in my_thread\n";
}
```

多线程

```
int main()
{
    MyThread x;    // 对x 的要求: 可复制
    thread th(x);  // 创建线程并执行
    thread th1(my_thread, 100);
    while(true)
    cout << "in main\n";
    return 0;
}
```

输出:

in my_thread
in my_thread
IN MYTHREAD
IN MYTHREAD
in main
in my_thread
IN MYTHREAD
IN MYTHREAD
in main
in main
in my_thread
IN MYTHREAD
.....



北京大学
PEKING UNIVERSITY

信息科学技术学院

C++ 14主要改进



二进制常量

```
cout << 0b1101 << endl;           //>>13
```

函数返回值自动推断

//C++14: 可以直接用 `auto` 当返回值, 编译器自动推类型

```
auto add(int a, int b) {  
    return a + b; // 自动推成 int  
}
```

auto lambda表达式

```
auto f = [](auto a, auto b) {  
    return a + b;  
};  
  
int main() {  
  
    cout << f(1, 2) << endl; //>>3  
    cout << f(1.1, 2.2) << endl; //>>4  
    return 0;  
}
```

auto lambda表达式

```
auto f = [](auto a, auto b) {  
    return a + b;  
};
```

//lambda表达式是语法糖。

//编译器自动生成的 匿名闭包类（伪代码）

```
struct __anonymous_closure_type {  
    template<typename T1, typename T2>  
        auto operator()(T1 a, T2 b) const {  
            return a + b;  
        }  
};
```

// f 就是这个闭包类的实例

```
__anonymous_closure_type f;
```


make_unique

```
std::unique_ptr<int> ptr =  
    std::make_unique<int>(20);
```

```
std::cout << *ptr << "\n"; //>>20
```



北京大学
PEKING UNIVERSITY

信息科学技术学院

C++ 17主要改进



类模板实参推导 CTAD

- 编译器根据**构造函数**的实参自动推导出模板类型(也适用于自定义的类模板)

```
std::pair p2(1, 1.2); // 编译器自动推导出 std::pair<int, double>
std::vector v3{1, 2, 3}; // 编译器自动推导出 std::vector<int>
```

```
struct Widget {
    void doSomething() {}
};
```

```
std::shared_ptr sp(new Widget); // 推导为 std::shared_ptr<Widget>
```

结构化绑定

`tuple`是c++11开始引入的可以存放任意多个不同类型的数据的序列

```
auto [id, name] = make_pair(1, "Alice");  
auto [a, b, c] = tuple{10, 3.14, "hello"}; //CTAD
```

// 解构结构体

```
struct Point { int x; int y; };  
Point p{10, 20};  
auto [x, y] = p;
```

tuple和结构化绑定

```
int main() {
    tuple<int, string, double> t{ 0, "hello", 3.14 };
    //C++ 17可接写  tuple t{ 0, "hello", 3.14};
    cout << get<0>(t) << endl;    // 输出0
    cout << get<1>(t) << endl;    // 输出hello
    // C++14支持按类型访问（需确保类型唯一）
    cout << get<double>(t) << endl;    // 输出3.14
    // C++17结构化绑定
    auto [a, b, c] = t;
    cout << a << " " << b << " " << c << endl;    // 输出0 hello 3.14
    get<0>(t) = 100;    //修改了t[0]
    auto & [x,y,z] = t;    //改x,y,z就是改t的元素
    x = 200;
    cout << get<0>(t) <<endl;    //输出200
    auto p = make_tuple(100,"s",2.3);
    cout << get<0>(p) << endl;
    return 0;
}
```

if/switch 初始化语句

```
if (auto it = mp.find(1); it != mp.end()) {  
    // 仅在此作用域使用 it  
}
```

```
switch (int val = getValue(); val) {  
    case 1: ...  
    case 2: ...  
}
```

折叠表达式 (Fold Expressions)

➤ 模板支持可变个数的类型参数将一个二元运算符应用到参数包的所有元素上

表格 		
特性	一元左折叠 (Unary Left Fold)	一元右折叠 (Unary Right Fold)
语法	<code>(... op pack)</code>	<code>(pack op ...)</code>
展开逻辑	从左向右依次结合	从右向左依次结合
数学形式	$((E_1 \text{ op } E_2) \text{ op } E_3) \dots$	$E_1 \text{ op } (E_2 \text{ op } (E_3 \dots))$

折叠表达式 (Fold Expressions)

➤ 一元折叠

```
template<typename ... Args>
auto sub(Args ... args) { //args是包, 即pack
    return (... - args); //一元左折叠
}
//sub(10,2,3,4) 展开为 ((10 -2) -3) -4)
template<typename ... T>
auto sub2(T ... x) { //x是pack
    return (x - ... ); //一元右折叠
}
//sub2(10,2,3,4) 展开为 (10-(2-(3-4)))
int main() {
    cout << sub(10,2,3,4) <<endl; //>>1
    cout << sub2(10,2,3,4) <<endl; //>>7
    return 0;
}
```


折叠表达式 (Fold Expressions)

➤ 一元折叠

```
template<class T>
void process(T v) {
    cout << "(" << v << ")";
}

template<typename... Args>
void process_all(Args... args) {
    (process(args), ...); //一元右折叠, 运算符op是逗号
    (... , process(args)); //一元左折叠, 运算符op是逗号
}

process_all(1, "ok", 2, "good"); //>>(1) (ok) (2) (good) (1) (ok) (2) (good)
```

左折叠的效果是 `process(1), ( process("ok"), (process(2), process("good")) )`
右折叠的效果是 `((process(1), process("ok")), process(2)), process("good")`

折叠表达式 (Fold Expressions)

➤ 二元折叠

语法: (init op ... op pack) 二元左折叠

语法: (pack op ... op init) 二元右折叠

```
template<typename... Args>
std::string format(Args... args) {
    // 二元左折叠: 初始值是 "Result: ", 操作符是 +
    return ("Result: " + ... + args); //二元左折叠
}
// 调用 format("A", "B")
// 展开为: (("Result: " + "A") + "B")
// 结果: "Result: AB"
```

折叠表达式 (Fold Expressions)

➤ 二元折叠

```
template<typename... Args>
void print(Args... args) {
    // 展开为: ((std::cout << arg1) << arg2) << arg3
    (std::cout << ... << args) << std::endl; //二元左折叠
}

print(1, "ok", 2, "good");           //>>1ok2good
```

折叠表达式 (Fold Expressions)

➤ 二元折叠

```
template<typename... Args>
void print2(Args&&... args) {
    (cout << ... << (cout << '*', args)) << endl;
}
print(1, "ok", 2, "good");           //>>*1*ok*2*good
```

本质上执行过程是：

```
cout << (cout<< '*', 1) << (cout<< '*', "ok") << (cout<< '*', 2)
```

(A,B)的计算过程是先计算A,然后返回B的值

折叠表达式 (Fold Expressions)

➤ 二元折叠

语法: (init op ... op pack)

```
template<typename... Args>
std::string format(Args... args) {
    // 二元左折叠: 初始值init是 "Result: ", 操作符是 +
    return ("Result: " + ... + args);
}

// 调用 format("A", "B")
// 展开为: ("Result: " + "A") + "B")
// 结果: "Result: AB"
```

std::optional —— 表示 “可能为空” 的值

➤ 替代返回特殊值 (如-1)

```
#include <iostream>
#include <optional>
using namespace std;
optional<int> func(int x) { // 返回值为int或空值
    if(x > 0) return x;
    return nullopt; // 空值
}
void print(int n) {
    auto res = func(n);
    if (res) cout << *res << endl;
    else     cout << "NULL" << endl;
}
int main() {
    print(20); //>>20
    print(-2); //>>NULL
    return 0;
}
```

std::any —— 安全存储任意类型的数据

```
#include <iostream>
#include <any>
#include <string>
using namespace std;
int main() {
    std::any val;
    if(val.has_value())
        cout << "has value" << endl; //不会输出
    val = 10;
    if (val.type() == typeid(int)) { //类型预判
        int n = std::any_cast<int>(val);
        cout << n << endl; //>>10
    }
    val = string("Hello");
    if (val.type() == typeid(string))
        cout << any_cast<string>(val); //>>Hello
    val = 3.14;
    return 0;
} 如果类型不对, any_cast会导致抛出 std::bad_any_cast 异常
```

std::any —— 安全存储任意类型的数据

```
std::map<std::string, std::any> config;  
config["Port"] = 8080;  
config["Host"] = std::string("127.0.0.1");  
config["Debug"] = true;  
config["Timeout"] = 30.5;
```

```
XXX key;
```

```
const auto& val = config[key];
```

```
// 根据类型分别处理
```

```
if (val.type() == typeid(int))
```

```
.....
```

```
else if (val.type() == typeid(std::string))
```

```
.....
```


std::variant —— 类型安全的联合体(union)

```
#include <variant>
#include <string>
#include <iostream>
using namespace std;
int main() {
    // 定义一个 variant, 它可以是 int, double 或 string
    variant<int, double, string> data;
    data = 10;
    data = 3.14;           //旧的 int 被销毁
    data = string("Hi");   // 存入 string
    string val = get<string>(data); //若类型不对则产生
std::bad_variant_access 异常
    cout << val << endl; //>>Hi
    return 0;
}
```

std::variant —— 类型安全的联合体(union)

```
int main() {
    variant<int, std::string> v = "Hello";
    cout << v.index() << endl; //>>1, 因为string是第1个类型
    if (std::holds_alternative<std::string>(v))
        //看里面是否是string对象
        std::cout << "Yes,it's a string" << endl; //会输出
    else
        std::cout << "not a string" << std::endl;
    v = 200;
    if (auto* pInt = std::get_if<int>(&v)) // 尝试获取 int 指针
        std::cout << "int value:" << *pInt << std::endl;
        //>>int value:200
    else //若v里面不是int,则pInt为nullptr
        cout << "not int" << endl;
    cout << v.index() << endl; //>>0, 因为int是第0个类型
    return 0;
}
```

std::variant —— 类型安全的联合体(union)

```
#include <iostream>
#include <variant>
#include <string>
using namespace std;
int main() {
    variant<int, double, std::string> data = 3.14;
    struct Visitor {
        void operator()(int i) const {
            std::cout << "Int: " << i << std::endl;
        }
        void operator()(const std::string& s) const {
            std::cout << "String: " << s << std::endl;
        }
    };
    visit(Visitor{}, data); //>>Int: 3    visit在variant中定义
    return 0;
} visit会调用data.index()然后根据结果调用 Visitor的不同 operator()
```

参数个数可变的模板

```
#include <tuple>
template<class ... Ts>
struct overloaded {
    std::tuple<Ts...> tp;
    overloaded(Ts... args) : tp(args...) { }
};
struct A { };
struct B { };
int main() {
    A a;
    B b;
    overloaded d(a, b); //d.tp 是tuple<A,B>类型
    return 0;
}
```

std::variant —— 类型安全的联合体(union)

```
// --- 这是一个 C++17 中实现重载 Lambda 的常用技巧 ---
template<class ... Ts> struct overloaded : Ts... {
    using Ts::operator()...; };
template<class ... Ts> overloaded(Ts...) -> overloaded<Ts...>;

int main() {
    std::variant<int, double, std::string> data = 3.14;
    std::visit(overloaded{ //传入一个临时 overloaded对象
        [](int i) { // 处理 int 类型
            std::cout << "int: " << i << std::endl;
        }, //lambda 表达式本质是函数对象类
        [](double d) { // 处理 double 类型
            std::cout << "float: " << d << std::endl;
        },
        [](const std::string& s) { // 处理 string 类型
            std::cout << "string: " << s << std::endl;
        }
    }, data); //>>float: 3.14
    return 0;
}
```

std::variant —— 类型安全的联合体(union)

```
template<class ... Ts>
struct overloaded : Ts... {
    using Ts::operator()...;
};
```

定义了一个类模版，名为 `overloaded`

`<class ... Ts>` 表示实例化模版时，可以有任意多个类型参数T1,T2,T3.....

`: Ts...` 表示overloaded有多个直接基类，分别是T1,T2,T3.....

`using Ts::operator()...;` 表示将从基类中继承得到的各个operator()函数都处理成重载函数

std::variant —— 类型安全的联合体(union)

```
template<class ... Ts> overloaded(Ts...) -> overloaded<Ts...>;
```

告诉编译器，overloaded的构造函数可以有任意多个实参，编译时根据构造函数的实参推断出多个类型参数 Ts...

本例中不写也可以

跨平台的文件系统库 std::filesystem

➤ 路径的拼接与解析

```
#include <iostream>
#include <filesystem>
namespace fs = std::filesystem;
int main() {
    // 1. 路径拼接: 自动适配系统分隔符
    fs::path base_dir = "/home/user"; // 在 Windows 上会自动处理为反斜杠
    fs::path file_path = base_dir / "documents" / "readme.txt";
    std::cout << "full path: " << file_path << std::endl;
    // 2. 路径分解
    std::cout << "file name: " << file_path.filename() << std::endl;
    // readme.txt
    std::cout << "extension: " << file_path.extension() << std::endl;
    // .txt
    std::cout << "parent dir: " << file_path.parent_path() << std::endl;
    // /home/user/documents
    return 0;
}
```


跨平台的文件系统库 std::filesystem

➤ 目录遍历

```
#include <iostream>    //要输出中文，文件要存为 ANSI格式
#include <filesystem>

namespace fs = std::filesystem;

int main() {
    fs::path dir_path = "."; // 当前目录
    std::cout << "--- 遍历当前目录 ---" << std::endl;
    // 1. 单层遍历
    for (const auto& entry : fs::directory_iterator(dir_path)) {
        // 判断是否为普通文件且后缀为 .txt
        if (entry.is_regular_file() &&
            entry.path().extension() == ".txt") {
            std::cout << "发现文本文件: " << entry.path().filename()
                << std::endl;
        }
    }
}
```

跨平台的文件系统库 std::filesystem

➤ 目录遍历

```
std::cout << "\n--- 递归遍历所有文件 ---" << std::endl;
// 2. 递归遍历 (包含子文件夹)
for(const auto& entry : fs::recursive_directory_iterator(dir_path)) {
    if (entry.is_regular_file()) {
        std::cout << "file path: " << entry.path() << std::endl;
    }
}
return 0;
}
```

跨平台的文件系统库 std::filesystem

➤ 文件操作与属性查询

```
include <iostream>
#include <filesystem>
#include <fstream>
namespace fs = std::filesystem;
int main() {
    fs::path target_file = "test_data/config.json";
    // 1. 检查是否存在
    if (!fs::exists(target_file)) {
        std::cout << "文件不存在, 正在创建目录和文件..." << std::endl;

        // 2. 创建多级目录 (类似 mkdir -p)
        fs::create_directories(target_file.parent_path());

        // 创建文件
        std::ofstream (target_file) << "{}";
    }
```

跨平台的文件系统库 std::filesystem

➤ 文件操作与属性查询

// 3. 获取文件属性

```
if (fs::is_regular_file(target_file)) {  
    auto size = fs::file_size(target_file);  
    auto write_time = fs::last_write_time(target_file);  
  
    std::cout << "文件大小: " << size << " 字节" << std::endl;  
    // 注意: last_write_time 返回的是 time_point,  
           // 通常需要转换才能打印具体时间  
}  
// 4. 复制文件  
fs::path backup = "test_data/config_backup.json";  
fs::copy(target_file, backup, fs::copy_options::overwrite_existing);  
return 0;  
}
```

std::apply: 把 tuple 解包传给函数

```
#include <iostream>
#include <tuple>
#include <string>
using namespace std;

int main() {
    auto my_tuple = make_tuple(10, "100.2", 3.14);
    apply([](int a, string b, double c) {
        double sum = a + stod(b) + c; //stod将字符串转double
        cout << sum << endl; //>>113.34
    }, my_tuple);
    return 0;
}
```

std::apply: 把 tuple 解包传给函数

➤ 编写函数模板处理未知元组

```
#include <iostream>
#include <tuple>
#include <string>
using namespace std;
// 一个泛型打印函数，可以处理任何类型的元组
template <typename TupleType>
void print_tuple(const TupleType& t) {
    std::apply([](const auto & ... args) { //使用折叠表达式 (C++17)
        // args... 是解包后的参数包
        (std::cout << ... << args) << std::endl;
        // 二元折叠表达式: ((cout << arg1) << arg2) ...
    }, t);
}
```

std::apply: 把 tuple 解包传给函数

➤ 编写函数模板处理未知元组

```
int main() {  
    std::tuple<int, double, std::string> t1(1, 3.14, "Hello ");  
    std::tuple<char, bool> t2('A', true);  
    print_tuple(t1); // 输出: 13.14Hello  
    print_tuple(t2); // 输出: A1  
    return 0;  
}
```

std::invoke: 统一调用函数 /lambda/ 成员函数

```
#include <iostream>
#include <functional> // std::invoke 的头文件
void normal_func(int x) {
    std::cout << "普通函数: " << x << std::endl;
}
class MyClass {
public:
    int value = 100;
    void member_func(int x) {
        std::cout << "成员函数: value + x = " << (value + x) << std::endl;
    }
};
int main() {
    std::invoke(normal_func, 10); // 相当于 normal_func(10);
    auto lambda = [](int a, int b) { return a + b; };
    int result = std::invoke(lambda, 3, 4); // 相当于 lambda(3, 4);
    std::cout << "Lambda 结果: " << result << std::endl;
```


std::invoke: 统一调用函数 /lambda/ 成员函数

```
MyClass obj;  
// 语法: std::invoke(成员函数指针, 对象(或指针), 参数...)  
// 相当于 obj.member_func(50);  
std::invoke(&MyClass::member_func, obj, 50);  
std::invoke(&MyClass::member_func, &obj, 50);  
// 语法: std::invoke(成员变量指针, 对象)  
// 相当于 obj.value;  
int val = std::invoke(&MyClass::value, obj);  
std::cout << "成员变量值: " << val << std::endl;  
return 0;  
}
```

用std::invoke实现万能执行器

```
#include <functional> // std::invoke
#include <string>
template <typename Callable, typename ... Args>
void my_executor(Callable&& func, Args&&... args) {
    //args里面每个类型前面都加 &&
    std::invoke(std::forward<Callable>(func),
               std::forward<Args>(args)...);
}
void say_hello(const std::string& name) {
    std::cout << "Hello, " << name << "!" << std::endl;
}
struct Robot {
    void work(int hours) {
        std::cout << "机器人正在工作 " << hours << " 小时。" << std::endl;
    }
};
```

用std::invoke实现万能执行器

```
int main() {
    my_executor(say_hello, "Alice"); //>>Hello, Alice!
    std::cout << "-----" << std::endl;
    Robot r;
    //传入成员函数指针，以及对象本身
    my_executor(&Robot::work, r, 8); //>>机器人正在工作 8 小时。
    std::cout << "-----" << std::endl;
    auto lambda = [](int x, int y,int z) {
        std::cout << "Lambda 结果: " << x + y + z << std::endl;
    };
    my_executor(lambda, 10, 20, 30);//>>Lambda 结果: 60
    return 0;
}
```